



Московский
авиационный
институт

национальный
исследовательский
университет

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

Автоматизированная платформа управления инфраструктурой с поддержкой подхода Architecture-as-Code

Направление: 01.03.02 «Прикладная математика и информатика»
Профиль: Информатика • Группа М8О-409Б-22

Москва • 2026

Студент: Концебалов Олег Сергеевич
Руководитель: Дзюба Дмитрий
Владимирович
Консультант: Булакина Мария Борисовна

ПОСТАНОВКА ЗАДАЧИ

Актуальность, цель и задача

Инфраструктура меняется быстрее документации: ручные правки, сбои и неполные rollout-процедуры приводят к drift между архитектурным описанием и фактической средой.

desired
state

≠
actual
state

Архитектура

Structurizr/C4-модель фиксирует требуемые host-level workload.

01

Фактическое

состояние

Inventory-сервис получает runtime-данные из cAdvisor и отмечает partial result.

02

Drift

Подтвержденное расхождение при полной actual data.

03

Цель

Разработать платформу, которая строит desired state, получает actual state, обнаруживает drift и запускает reconcile.

04

Вывод: работа переводит архитектурное описание из статичной документации в источник данных для control loop.

Существующие решения закрывают части задачи

Для ВКР важен легковесный контур:
Architecture-as-Code → inventory →
drift detection → reconcile.

Ansible

Сильная сторона:
agentless, зрелая
экосистема

Ограничение:
job-based запуск без
встроенной непрерывной
реконсильации

Terraform / Pulumi

Сильная сторона:
декларативное описание
ресурсов

Ограничение:
фокус на provisioning и
state-файл, не на host-level
workload

K8s / GitOps

Сильная сторона:
сильная control-loop
модель

Ограничение:
естественная область —
Kubernetes API и
кластерные ресурсы

Проект ВКР

Сильная сторона:
Architecture-as-Code как
source of truth

Ограничение:
легковесный MVP для
Docker-based workload вне
Kubernetes

Вывод: собственная платформа нужна не вместо этих инструментов, а как связующий control plane для выбранного сценария.

Ключевые требования к системе

Требования сформулированы вокруг безопасного цикла управления состоянием инфраструктуры.

Функциональные

- загрузка Structurizr JSON
- построение host-centric desired state
- сбор actual state через cAdvisor
- обнаружение отсутствующих workload
- запуск reconcile для node_exporter и cadvisor

F

Архитектурные

- четыре независимых сервиса
- явные HTTP API-контракты
- read-only snapshot для state-сервисов
- registry detector-ов и operator-ов
- контейнеризация и внешний конфиг

A

Эксплуатационные

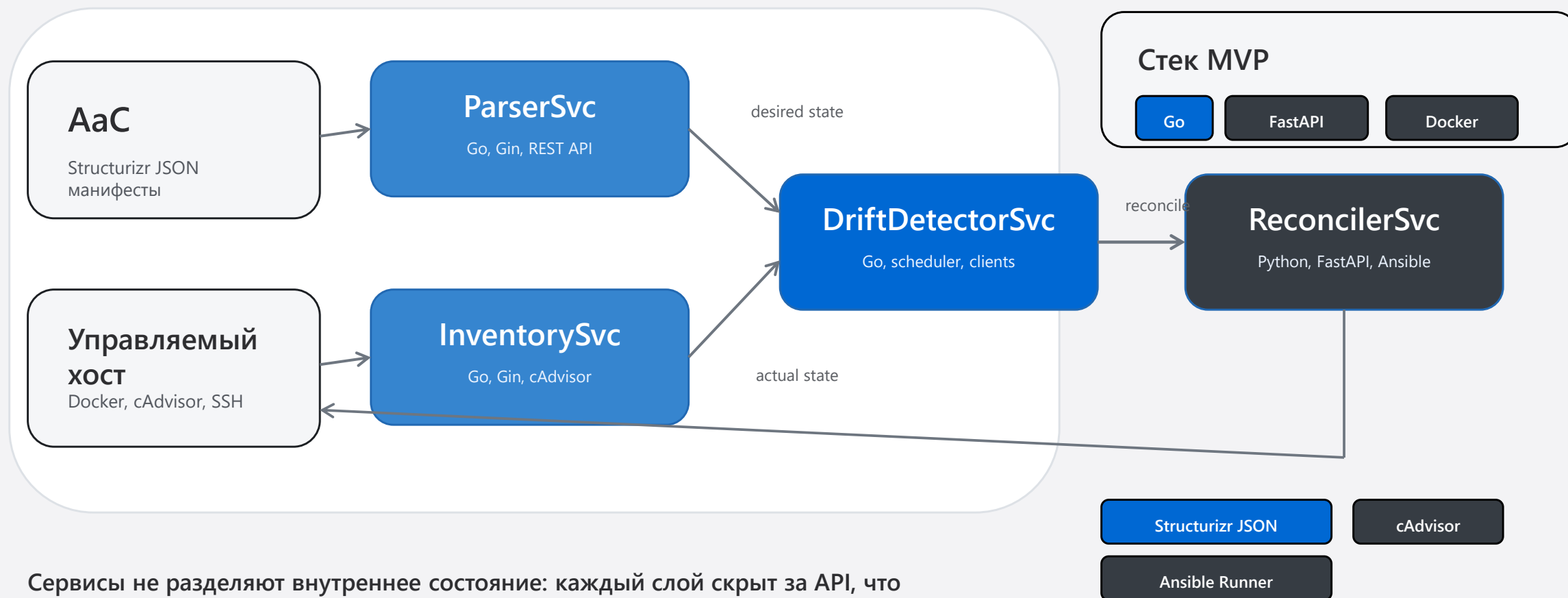
- partial data не считается drift
- cooldown подавляет повторные команды
- асинхронная очередь reconcile
- ограничение параллелизма worker-ов
- SSH-параметры через окружение

O

Ключевое следствие: платформа автоматизирует только подтвержденные и заранее поддерживаемые действия восстановления.

Архитектура решения и стек технологий

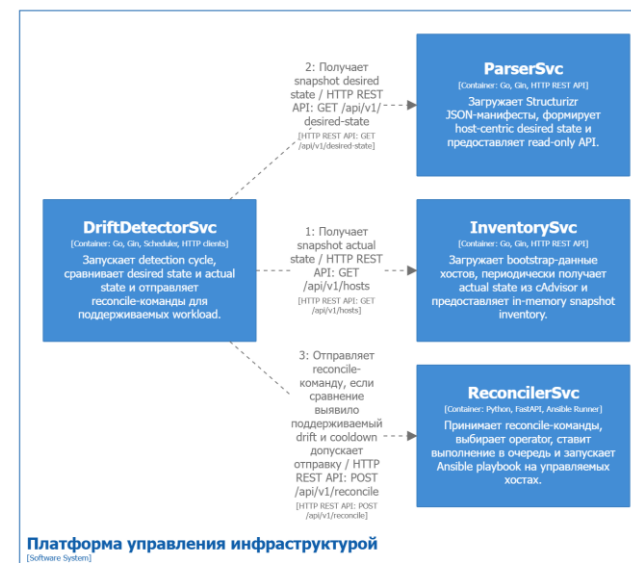
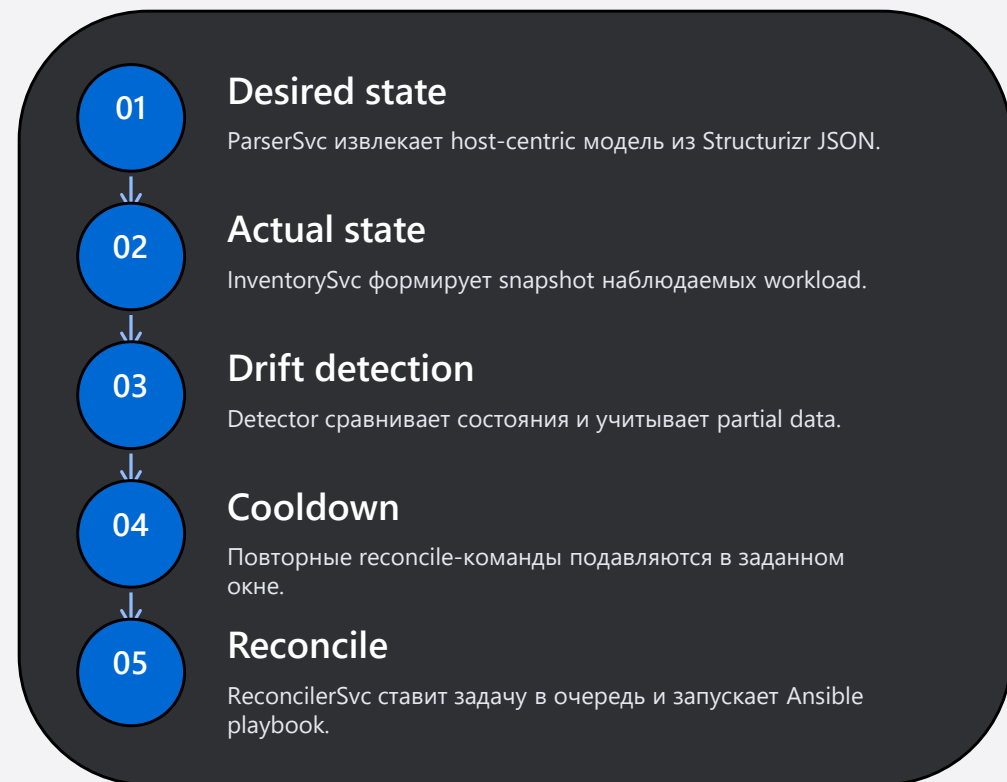
Упрощённая схема построена по C4-диаграммам проекта и реальным сервисным контрактам.



Сервисы не разделяют внутреннее состояние: каждый слой скрыт за API, что снижает связность и упрощает развитие.

Control loop: desired → actual → drift → reconcile

Вместо ML-модели в работе используется модель согласования состояния инфраструктуры.



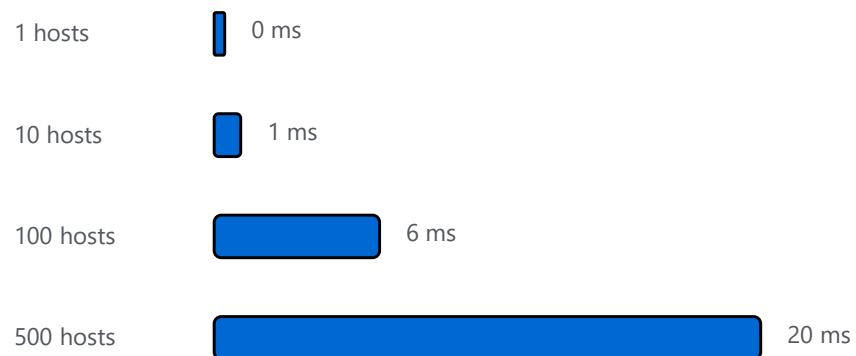
Реальная динамическая диаграмма сценария обнаружения drift из материалов проекта

Ключевая гарантия MVP: отсутствие данных не является основанием для автоматического восстановления.

Проверка качества разработанного решения

Использованы unit/component tests и E2E benchmark harness поверх локального compose-стенда.

Detection scalability, p50



Рост цикла в steady-state связан в основном с fetch-стадиями Inventory и Parser.

14.3 s

сходимость после
drift node_exporter

102 / 180

accepted при saturation
probe

1.0

cooldown
suppression ratio

Покрывтие

тестами

Unit/component tests по
четырем сервисам.
E2E harness: 5 benchmark-
сценариев.

Partial data

20 host, 6 недоступных FQDN.
cycle.partial=true, failedHosts=6,
reconcile для missing hosts=0.

Вывод: MVP подтверждает работоспособность control loop и явно показывает границы производительности и безопасности.

РЕЗУЛЬТАТЫ РАБОТЫ

Результаты разработки MVP

Архитектурное описание, inventory, drift detection и reconcile связаны в единый программный цикл.

ParserSvc

desired state
Structurizr JSON

InventorySvc

actual state
partial metadata

DriftDetectorSvc

detectors
cooldown

ReconcilerSvc

operators
Ansible Runner



QR на GitHub

github.com/BaronPipistron/Infrastructure-Management-Platform

Практически й результат

- Docker Compose E2E
- node_exporter и cadvisor
- benchmark harness
- ограничения MVP зафиксированы

Architecture-as-Code в эксплуатационном контуре

Результат ВКР — проверенная декомпозиция будущего internal control plane для host-level workload.

Ценность для команды

- меньше ручных сверок
- архитектурная модель используется машинно
- drift обнаруживается до инцидента
- восстановление через operator-ы

Расширяемость

- новые actual-state sources: NetBox, CMDB, Prometheus
- новые detector-ы и operator-ы
- version/config/health drift
- persistence истории и job state

Границы MVP

- in-memory state
- два workload
- нет authN/authZ и audit trail
- нет API статуса reconcile-задачи
- результаты benchmark не являются SLA

Итог: цель ВКР достигнута — реализован и проверен базовый цикл desired state → actual state → drift → reconcile.